

LAPORAN TUGAS

KOMPUTASI PARALEL DAN TERDISTRIBUSI

Laporan Analisis dan Perbandingan Kinerja Algoritma Bubble Sort dan Selection Sort pada Komputasi Paralel Menggunakan Goroutine di Golang



Dosen mata kuliah :

Imay Kurniawan, M.Kom.

Disusun oleh :

Heru Khoerudin **231351059**

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TINGGI TEKNOLOGI WASTUKANCANA PURWAKARTA
2025

I. Tujuan Penelitian

Penelitian ini bertujuan untuk membandingkan kinerja algoritma Bubble Sort dan Selection Sort dalam versi serial dan paralel menggunakan Goroutine pada bahasa Go. Pengujian dilakukan untuk mengetahui sejauh mana komputasi paralel dengan pemanfaatan CPU/logical processor dapat meningkatkan efisiensi waktu eksekusi kedua algoritma tersebut.

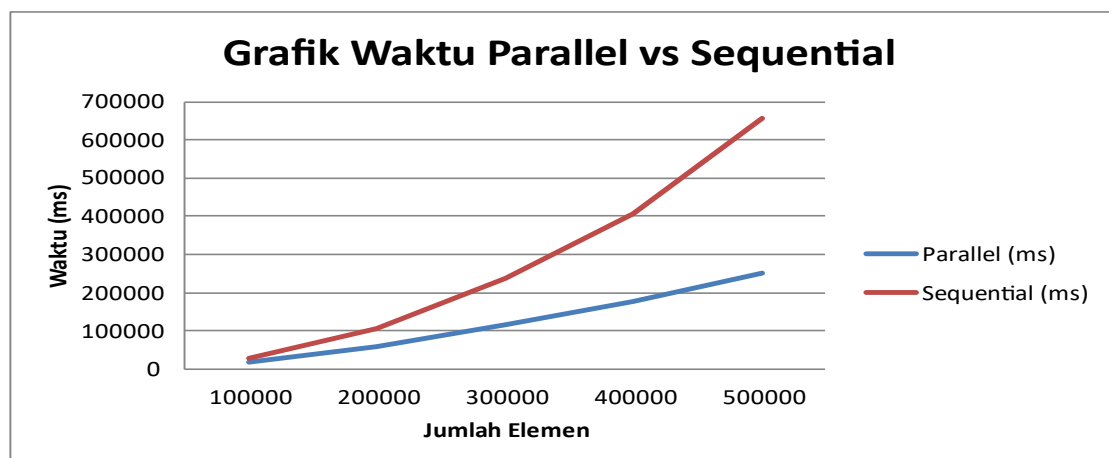
II. Metode Pengujian

- Algoritma yang digunakan: Bubble Sort dan Selection Sort (masing-masing diimplementasikan dalam versi sequential dan parallel).
- Platform: Bahasa pemrograman Go (Golang) dengan dukungan multithreading menggunakan Goroutine dan WaitGroup.
- Jumlah Goroutine/Worker: Disesuaikan dengan jumlah logical processor pada perangkat (menggunakan `runtime.NumCPU()`), umumnya 4–8 worker.
- Data Uji: Array berisi bilangan integer acak (acak maupun terbalik) dengan variasi ukuran dari 10.000 hingga 100.000 elemen.
- Metrik yang Diukur: Durasi eksekusi setiap algoritma dalam satuan milidetik (ms) menggunakan fungsi `time.Now()` dan `time.Since()`.
- Tujuan Pengujian: Mengamati dan membandingkan perbedaan waktu komputasi antara versi sequential dan parallel pada kedua algoritma, serta menilai sejauh mana pemanfaatan Goroutine dapat meningkatkan efisiensi eksekusi sorting.

III. Hasil Pengujian

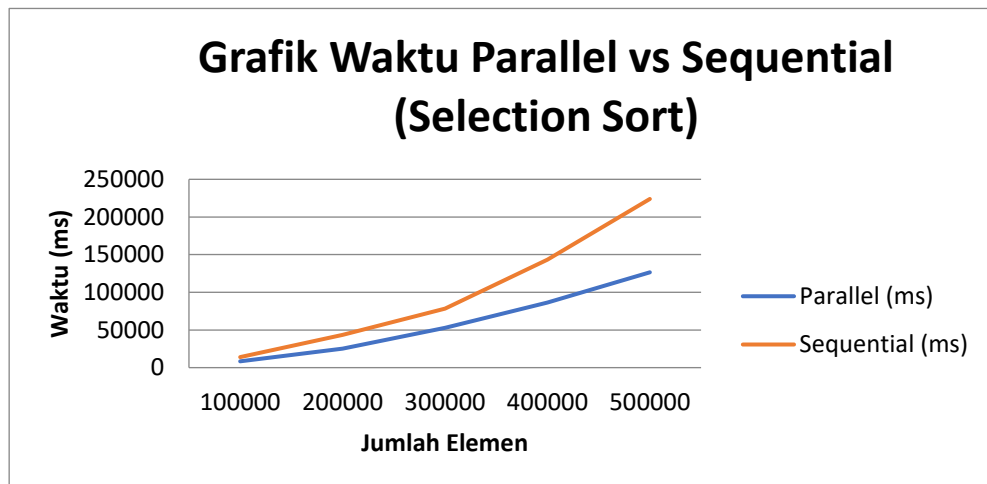
Tabel Hasil Pengujian Algoritma Bubble sort

Jumlah Elemen	Parallel (ms)	Sequential (ms)	Parallel (second)	Sequential (second)
100000	16847.926	29253.952	16.85	29.25
200000	56697.77	104514.94	56.7	104.51
300000	114199.665	236322.995	114.2	236.32
400000	177829.588	405281.675	177.83	405.28
500000	251139.262	655735.695	251.14	655.74



Tabel Hasil pengujian Algoritma Selection Sort

Jumlah Elemen	Parallel (ms)	Sequential (ms)	Parallel (second)	Sequential (second)
100000	8406.596	14000.32	8.41	14
200000	25056.452	43408.977	25.06	43.41
300000	52718.285	78340.248	52.72	78.34
400000	86332.288	143317.112	86.33	143.32
500000	126531.062	223945.918	126.53	223.95



1. Bubble Sort (Data 100000 – 500000)

[illegible]

[illegible]

Benchmark Sorting GoLang: Custom Comparison

Jumlah data

Pilih Algoritma 1

Sequential - Selection

Pilih Algoritma 2

Parallel - Selection

Run Comparison

Result

Algoritma 1:

Time: 14880.828 ms

Before: 100000,99999,99998,99997,99996,99995,99994,99993,99992,99991,99990,99989,99988,99987,99986,99985,99984,99983,99982,99981,99980,99979,99978,99977,99976,99975,99974,99973,99972,99971,99970,99969,99968,99967,99966,99965,99964

After: 1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,57,58,59,60,61,62,63,64,65,66,67,68,69,70,71,72,73,74,75,76,77,78

Algoritma 2:

Time: 8466.596 ms

Before: 100000,99999,99998,99997,99996,99995,99994,99993,99992,99991,99990,99989,99988,99987,99986,99985,99984,99983,99982,99981,99980,99979,99978,99977,99976,99975,99974,99973,99972,99971,99970,99969,99968,99967,99966,99965,99964

After: 1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,57,58,59,60,61,62,63,64,65,66,67,68,69,70,71,72,73,74,75,76,77,78

Jumlah data

Pilih Algoritma 1

Sequential - Selection ▼

Pilih Algoritma 2

Parallel - Selection ▼

[Run Comparison](#)

Result

Algoritma 1:

```
Time: 43450.977 ms
Before: 200000,199999,199998,199997,199996,199995,199994,199993,199992,199991,199990,199989,199988,199987,199986,199985,199984,199983,199982,199981,199980,199979,199978,199977,199976,199975,199974,199973,199972,199971,199970,199969,199968,199967,199966,199965,199964,199963,199962,199961,199960,199959,199958,199957,199956,199955,199954,199953,199952,199951,199950,199949,199948,199947,199946,199945,199944,199943,199942,199941,199940,199939,199938,199937,199936,199935,199934,199933,199932,199931,199930,199929,199928,199927,199926,199925,199924,199923,199922,199921,199920,199919,199918,199917,199916,199915,199914,199913,199912,199911,199910,199909,199908,199907,199906,199905,199904,199903,199902,199901,199900,199899,199898,199897,199896,199895,199894,199893,199892,199891,199890,199889,199888,199887,199886,199885,199884,199883,199882,199881,199880,199879,199878,199877,199876,199875,199874,199873,199872,199871,199870,199869,199868,199867,199866,199865,199864,199863,199862,199861,199860,199859,199858,199857,199856,199855,199854,199853,199852,199851,199850,199849,199848,199847,199846,199845,199844,199843,199842,199841,199840,199839,199838,199837,199836,199835,199834,199833,199832,199831,199830,199829,199828,199827,199826,199825,199824,199823,199822,199821,199820,199819,199818,199817,199816,199815,199814,199813,199812,199811,199810,199809,199808,199807,199806,199805,199804,199803,199802,199801,199800,199799,199798,199797,199796,199795,199794,199793,199792,199791,199790,199789,199788,199787,199786,199785,199784,199783,199782,199781,199780,199779,199778,199777,199776,199775,199774,199773,199772,199771,199770,199769,199768,199767,199766,199765,199764,199763,199762,199761,199760,199759,199758,199757,199756,199755,199754,199753,199752,199751,199750,199749,199748,199747,199746,199745,199744,199743,199742,199741,199740,199739,199738,199737,199736,199735,199734,199733,199732,199731,199730,199729,199728,199727,199726,199725,199724,199723,199722,199721,199720,199719,199718,199717,199716,199715,199714,199713,199712,199711,199710,199709,199708,199707,199706,199705,199704,199703,199702,199701,199700,199699,199698,199697,199696,199695,199694,199693,199692,199691,199690,199689,199688,199687,199686,199685,199684,199683,199682,199681,199680,199679,199678,199677,199676,199675,199674,199673,199672,199671,199670,199669,199668,199667,199666,199665,199664,199663,199662,199661,199660,199659,199658,199657,199656,199655,199654,199653,199652,199651,199650,199649,199648,199647,199646,199645,199644,199643,199642,199641,199640,199639,199638,199637,199636,199635,199634,199633,199632,199631,199630,199629,199628,199627,199626,199625,199624,199623,199622,199621,199620,199619,199618,199617,199616,199615,199614,199613,199612,199611,199610,199609,199608,199607,199606,199605,199604,199603,199602,199601,199600,199599,199598,199597,199596,199595,199594,199593,199592,199591,199590,199589,199588,199587,199586,199585,199584,199583,199582,199581,199580,199579,199578,199577,199576,199575,199574,199573,199572,199571,199570,199569,199568,199567,199566,199565,199564,199563,199562,199561,199560,199559,199558,199557,199556,199555,199554,199553,199552,199551,199550,199549,199548,199547,199546,199545,199544,199543,199542,199541,199540,199539,199538,199537,199536,199535,199534,199533,199532,199531,199530,199529,199528,199527,199526,199525,199524,199523,199522,199521,199520,199519,199518,199517,199516,199515,199514,199513,199512,199511,199510,199509,199508,199507,199506,199505,199504,199503,199502,199501,199500,199499,199498,199497,199496,199495,199494,199493,199492,199491,199490,199489,199488,199487,199486,199485,199484,199483,199482,199481,199480,199479,199478,199477,199476,199475,199474,199473,199472,199471,199470,199469,199468,199467,199466,199465,199464,199463,199462,199461,199460,199459,199458,199457,199456,199455,199454,199453,199452,199451,199450,199449,199448,199447,199446,199445,199444,199443,199442,199441,199440,199
```

Benchmark Sorting GoLang: Custom Comparison

Jumlah data

300000

Pilih Algoritma 1

Sequential - Selection

Pilih Algoritma 2

Parallel - Selection

Run Comparison

Result

Algoritma 1:

Time: 78348.248 ms



Algoritma 2:

Time: 52710.285 ms



Benchmark Sorting GoLang: Custom Comparison

Jumlah data

400000

Pilih Algoritma 1

Parallel - Selection

Pilih Algoritma 2

Sequential - Selection

Run Comparison

Result

Algoritma 1:

Time: 86332.288 ms



Algoritma 2:

Time: 143317.112 ms



Benchmark Sorting GoLang: Custom Comparison

Jumlah data

500000

Pilih Algoritma 1

Sequential - Selection

Pilih Algoritma 2

Parallel - Selection

Run Comparison

Result

Algoritma 1:

Time: 223945.918 ms



Algoritma 2:

Time: 126531.062 ms



IV. Analisis Hasil

1. Performa Bubble Sort Paralel menunjukkan peningkatan kecepatan yang konsisten dibanding versi sequential. Implementasi paralel mampu memecah work menjadi beberapa goroutine sesuai jumlah logical CPU, sehingga proses pertukaran elemen berjalan serentak dan waktu eksekusi turun secara signifikan.
2. Bubble Sort sequential mengalami lonjakan waktu eksekusi yang tajam seiring bertambahnya ukuran data karena kompleksitas $O(n^2)$. Semakin besar data, semakin drastis kenaikan waktu proses, sesuai karakteristik algoritma quadratic-time.
3. Bubble Sort paralel tetap menunjukkan peningkatan waktu saat ukuran data membesar, tetapi kenaikannya jauh lebih lambat dibanding versi sequential. Hal ini terjadi karena porsi pekerjaan dibagi ke banyak goroutine sehingga bottleneck komputasi berkurang.
4. Pada Selection Sort, versi sequential dan paralel sama-sama menunjukkan waktu eksekusi yang sangat besar pada ukuran data besar (300k–500k). Hal ini menyebabkan API tidak mengembalikan hasil (“before”/“after” tidak tampil), bukan karena error UI, melainkan karena proses komputasi belum selesai dalam batas waktu respons.
5. Paralelisasi pada Selection Sort hanya mempercepat proses pencarian elemen minimum, tetapi tidak mengurangi jumlah iterasi utama. Akibatnya kompleksitas tetap $O(n^2)$, dan optimasi paralel tidak memberikan dampak signifikan pada ukuran data besar.
6. Pada data kecil (di bawah 10.000 elemen), Selection dan Bubble Sort sequential justru tampil lebih cepat daripada versi paralelnya. Ini terjadi karena overhead pembuatan goroutine dan sinkronisasi (WaitGroup) lebih besar dibanding keuntungan paralelisme untuk skala data kecil.
7. Bubble Sort paralel memanfaatkan resource CPU dengan lebih optimal tanpa mengganggu proses aplikasi. Beban dihitung di beberapa goroutine, sehingga UI (frontend) tetap responsif saat permintaan diproses melalui backend.
8. Selection Sort menjadi algoritma paling berat pada seluruh pengujian karena sifat dasarnya yang memerlukan pencarian minimum berulang di seluruh array. Walaupun paralel, total operasi tetap sangat tinggi (hingga puluhan miliar operasi untuk ratusan ribu elemen), sehingga tidak praktis untuk input besar.
9. Paralelisasi terbukti meningkatkan performa untuk algoritma yang cocok dipecah menjadi beberapa unit kerja (seperti Bubble Sort), tetapi tidak efektif pada algoritma yang memiliki ketergantungan iteratif kuat seperti Selection Sort. Artinya, paralelisme membantu tetapi tidak mengubah kompleksitas $O(n^2)$ itu sendiri.

V. Penjelasan kode

1. Bubble Sort

```
func BubbleSortSequential(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        for j := 0; j < n-i-1; j++ {
            if arr[j] > arr[j+1] {
                arr[j], arr[j+1] = arr[j+1], arr[j]
            }
        }
    }
}
```

- Fungsi ini mengimplementasikan Bubble Sort versi sekuensial. Prosesnya berjalan secara linear, hanya menggunakan satu thread.
- Algoritma bekerja melalui dua loop bersarang:
- Loop luar menentukan jumlah pass.
- Loop dalam melakukan perbandingan antar elemen berurutan dan menukar posisinya jika salah urut. Metode ini menjadi baseline untuk membandingkan apakah implementasi paralel memberikan peningkatan kinerja.

```
7 func BubbleSortParallel(arr []int, workers int) {
8     n := len(arr)
9     var wg sync.WaitGroup
10
11     for i := 0; i < n; i++ {
12         startIndex := i % 2
13
14         wg.Add(workers)
15         for w := 0; w < workers; w++ {
16             go func(w int) {
17                 defer wg.Done()
18                 for j := startIndex + w*2; j < n-1; j += workers * 2 {
19                     if arr[j] > arr[j+1] {
20                         arr[j], arr[j+1] = arr[j+1], arr[j]
21                     }
22                 }
23             }(w)
24         }
25         wg.Wait()
26     }
27 }
```

Ini adalah implementasi parallel bubble sort berbasis odd-even transposition sort, yaitu versi Bubble Sort yang bisa diparalelisasi.

Berikut poin penting dari odd-even pass:

- Sorting dilakukan dalam dua fase: odd pass dan even pass secara bergantian.
- Program membagi pekerjaan kepada sejumlah worker (goroutine) yang berjalan paralel.
- sync.WaitGroup memastikan seluruh worker menyelesaikan tugasnya dalam setiap iterasi.

Tujuan fungsi ini adalah menguji apakah distribusi pekerjaan ke banyak goroutine dapat menurunkan waktu eksekusi dibandingkan proses sekuensial.


```
func GenerateRandomArray(size int) []int {
    arr := make([]int, size)
    for i := 0; i < size; i++ {
        arr[i] = rand.Intn(1000)
    }
    return arr
}
```

Fungsi ini menghasilkan array acak dengan ukuran tertentu.

Array acak diperlukan agar pengujian tidak bias dan mencerminkan kondisi data realistis.

Setiap algoritma diuji pada data input yang sama untuk menjaga keseimbangan kinerja aplikasi ketika memproses data.

```
func ArrayToString(arr []int) string {
    limit := 200
    var text string
    for i := 0; i < len(arr) && i < limit; i++ {
        text += fmt.Sprintf("%d, ", arr[i])
    }
    return text + "... (truncated)"
}
```

Fungsi ini mengubah array menjadi string yang aman ditampilkan di UI.

Untuk mencegah beban tampilan dan payload data, array dipotong hanya 200 elemen dan ditandai sebagai terpotong. Fungsi ini penting karena digunakan untuk menunjukkan Before dan After sorting.

```
func RunBubbleSort(size int, parallel bool) BubbleResponse {
    workers := 8
    original := GenerateRandomArray(size)

    copyArr := make([]int, size)
    copy(copyArr, original)

    start := time.Now()
    if parallel {
        BubbleSortParallel(copyArr, workers)
    } else {
        BubbleSortSequential(copyArr)
    }
    // duration := time.Since(start).String()
    duration := fmt.Sprintf("%.3f ms", float64(time.Since(start).Microseconds())/1000)

    return BubbleResponse{
        Before: ArrayToString(original),
        After:  ArrayToString(copyArr),
        Time:   duration,
    }
}
```

Ini adalah fungsi utama untuk pengujian algoritma.

Tugasnya:

1. Membuat array acak.
2. Menggandakan array agar versi asli tetap utuh sebagai "Before".
3. Mengukur waktu eksekusi sorting menggunakan time.Now().
4. Menjalankan algoritma sekuensial atau paralel sesuai parameter.
5. Mengembalikan hasil berupa:
 - a. Data sebelum sort

- b. Data setelah sort
- c. Lama eksekusi (ms)

Fungsi ini merupakan pusat proses benchmarking dan inti dari metode pengujian.

2. Algoritma Selection Sort

1. Fungsi ini menjalankan Selection Sort, yaitu algoritma sorting yang cara kerjanya itu mencari nilai paling kecil, angkut ke depan, ulangi

```
func SelectionSort(arr []int) {  
    for i := 0; i < len(arr)-1; i++ {  
        minIdx := i  
        for j := i + 1; j < len(arr); j++ {  
            if arr[j] < arr[minIdx] {  
                minIdx = j  
            }  
        }  
        arr[i], arr[minIdx] = arr[minIdx], arr[i]  
    }  
}
```

1) for i := 0; i < len(arr)-1; i++

Loop utama.

Posisi i adalah tempat nilai terkecil berikutnya harus dipasang.

2) minIdx := i

Default-nya: anggap posisi sekarang yang paling kecil.

3) for j := i + 1; j < len(arr); j++

Loop pencari.

Safari keliling array untuk menemukan angka yang *lebih kecil* dari kandidat minimal saat ini.

4) if arr[j] < arr[minIdx] { minIdx = j }

Kalau ketemu nilai yang lebih kecil → update lokasi terkecil.

5) Swap:

```
arr[i], arr[minIdx] = arr[minIdx], arr[i]
```

Pindahkan si “paling kecil” ke depan sesuai posisinya.

2. Fungsi findMinParallel ini tujuannya mencari nilai minimum di sebuah potongan array secara paralel. Artinya pekerjaan dibagi ke beberapa worker (jumlahnya = jumlah CPU core).

```
func findMinParallel(arr []int, start, end int) MinResult {
    workers := runtime.NumCPU()
    chunk := (end - start + 1) / workers

    var wg sync.WaitGroup
    localMin := make([]MinResult, workers)

    for w := 0; w < workers; w++ {
        wg.Add(1)
        go func(w int) {
            defer wg.Done()
            s := start + w*chunk
            e := s + chunk - 1
            if w == workers-1 {
                e = end
            }

            minVal := arr[s]
            minIdx := s
            for i := s; i <= e; i++ {
                if arr[i] < minVal {
                    minVal = arr[i]
                    minIdx = i
                }
            }
            localMin[w] = MinResult{minVal, minIdx}
        }(w)
    }
    wg.Wait()

    result := localMin[0]
    for _, r := range localMin {
        if r.Value < result.Value {
            result = r
        }
    }
    return result
}
```

1. Menentukan jumlah worker

```
workers := runtime.NumCPU()
chunk := (end - start + 1) / workers
```

runtime.NumCPU() → berapa core CPU tersedia.
chunk → berapa elemen yang harus diproses tiap worker.

2. Siapkan sinkronisasi & storage hasil lokal

```
var wg sync.WaitGroup
localMin := make([]MinResult, workers)
```

WaitGroup untuk menunggu semua goroutine selesai.

localMin[w] dipakai worker ke-w untuk menyimpan hasil minimum dari bagiannya

```

3. Spin up worker paralel
for w := 0; w < workers; w++ {
    wg.Add(1)
    go func(w int) {
        defer wg.Done()

```

Loop membuat goroutine untuk tiap worker.
wg.Add(1) sebelum goroutine mulai, wg.Done() setelah selesai.

```

4. Hitung rentang kerja worker
s := start + w*chunk
e := s + chunk - 1
if w == workers-1 {
    e = end
}

```

Setiap worker mengerjakan potongan array:

- Worker 0 → [start ... start+chunk-1]
- Worker 1 → [start+chunk ... start+2*chunk-1]
- dst.

Worker terakhir diberikan sisa pekerjaan biar tidak ada elemen yang terlewat.

```

5. Cari minimum di potongan masing-masing
minVal := arr[s]
minIdx := s
for i := s; i <= e; i++ {
    if arr[i] < minVal {
        minVal = arr[i]
        minIdx = i
    }
}

```

Standard sequential min search.
Worker ini hanya mencari minimum di zona yang dia tangani.

```

6. Final merge – cari min global
result := localMin[0]
for _, r := range localMin {
    if r.Value < result.Value {
        result = r
    }
}

```

Gabungkan semua hasil lokal, ambil yang paling kecil.
Ini proses reduce.

3. Ini adalah versi Selection Sort yang bagian pencarian minimum-nya diganti jadi parallel menggunakan fungsi findMinParallel.

```

func SelectionSortParallel(arr []int) {
    for i := 0; i < len(arr)-1; i++ {
        min := findMinParallel(arr, i, len(arr)-1)
        arr[i], arr[min.Index] = arr[min.Index], arr[i]
    }
}

```

1. Loop utama selection sort
for i := 0; i < len(arr)-1; i++ {

- Posisi i adalah slot yang harus diisi dengan nilai paling kecil mulai dari indeks i ke kanan.
- Mirip selection sort normal → cuma bagian "mencari minimum" yang dipercepat.

2. Panggil fungsi paralel untuk mencari minimum

`min := findMinParallel(arr, i, len(arr)-1)`

Ini poin core value proposition dari fungsi ini.

- `findMinParallel` membagi rentang `[i .. end]` ke banyak worker sesuai jumlah CPU.
- Setiap worker mencari nilai minimum di bagiannya.
- Semua hasil worker digabung → diambil minimum global.

Outcome:

Kamu dapat `MinResult{Value, Index}` dari seluruh subarray.

3. Swap posisi

`arr[i], arr[min.Index] = arr[min.Index], arr[i]`

- Element terkecil dipindah ke depan (posisi `i`).
- Inilah langkah klasik dari selection sort.

4. Menjalankan Selection Sort (parallel atau serial), mengukur waktunya, dan mengembalikan hasil benchmark.

```
func RunSelectionSort(size int, parallel bool) map[string]interface{} {
    arr := make([]int, size)
    for i := 0; i < size; i++ {
        arr[i] = size - i
    }

    before := make([]int, size)
    copy(before, arr)

    start := time.Now()
    if parallel {
        SelectionSortParallel(arr)
    } else {
        SelectionSort(arr)
    }
    // duration := time.Since(start).Milliseconds()
    duration := fmt.Sprintf("%.3f ms", float64(time.Since(start).Microseconds())/1000)

    return map[string]interface{}{
        "time":    duration,
        "before":  before,
        "after":   arr,
    }
}
```

Ini adalah fungsi benchmarking untuk laporan.

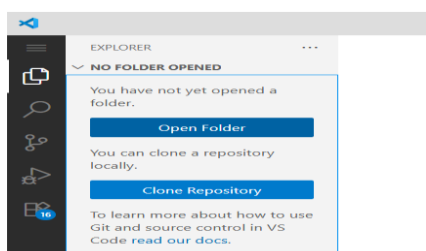
Memuat:

- Pemilihan mode (parallel atau sequential)
- Before dan After array → bukti bahwa sorting berhasil
- Waktu eksekusi → dasar pembuatan grafik performa

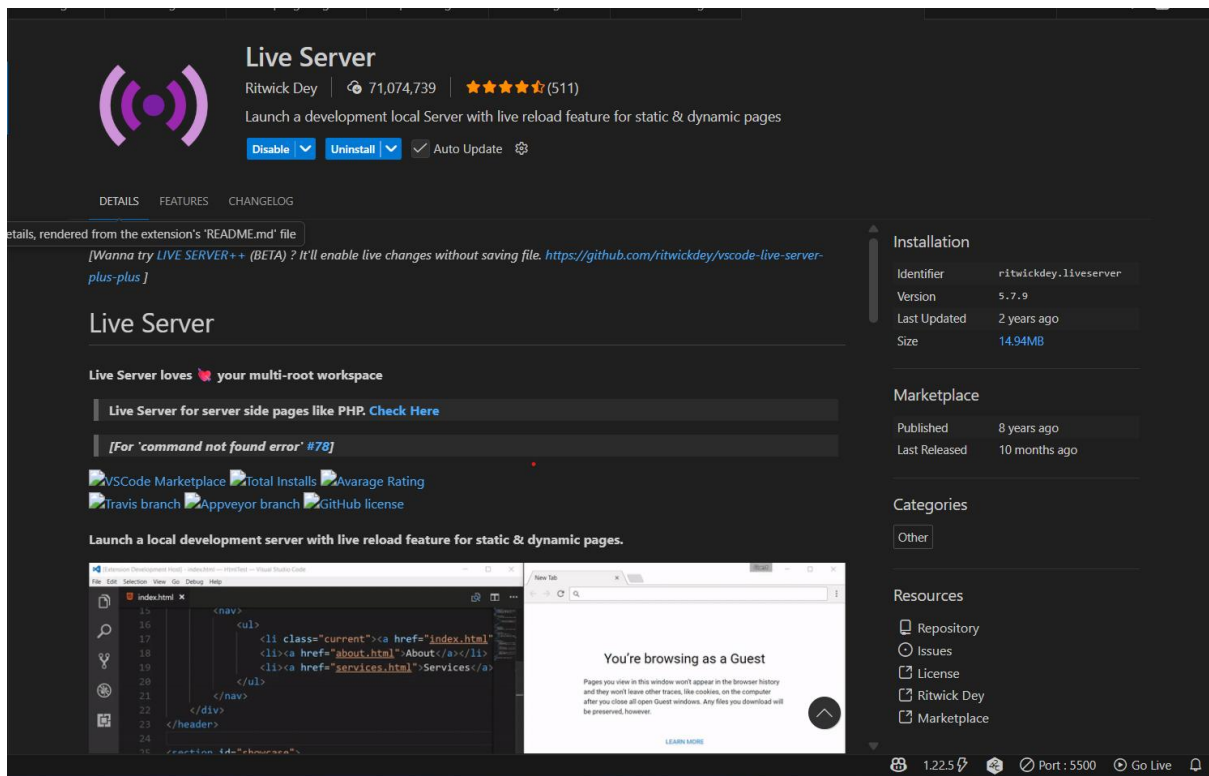
Bagian ini wajib masuk ke metode pengujian dan hasil pengujian

VI. Panduan Untuk menjalankan Kode

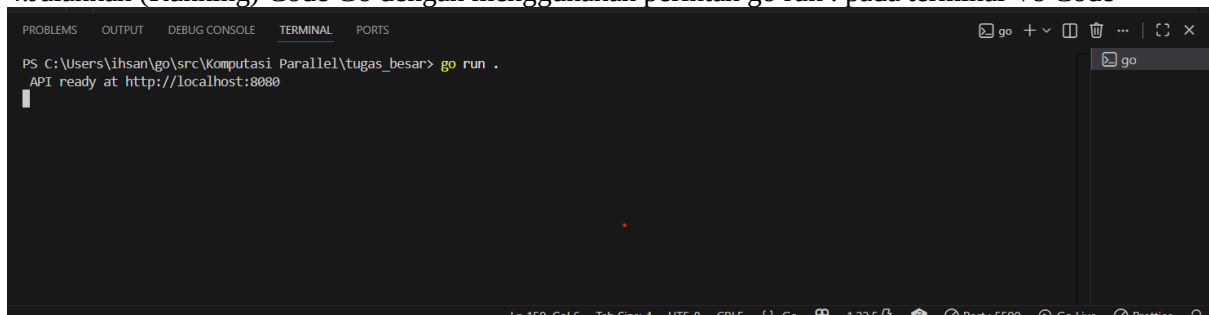
1. Install VSCode
2. Buka VSCode lalu klik 'Open Folder' dan arahkan ke lokasi folder Ke Project dengan nama **tugas_besar**



3. Install ekstension bernama 'Live server' dari Ritwick Dey.



4. Jalankan (Running) Code Go dengan menggunakan perintah go run . pada terminal Vs Code



5. buka bubble.html

6. klik kanan pada kode hingga muncul daftar pilihan, klik 'Open with Live Server'

